

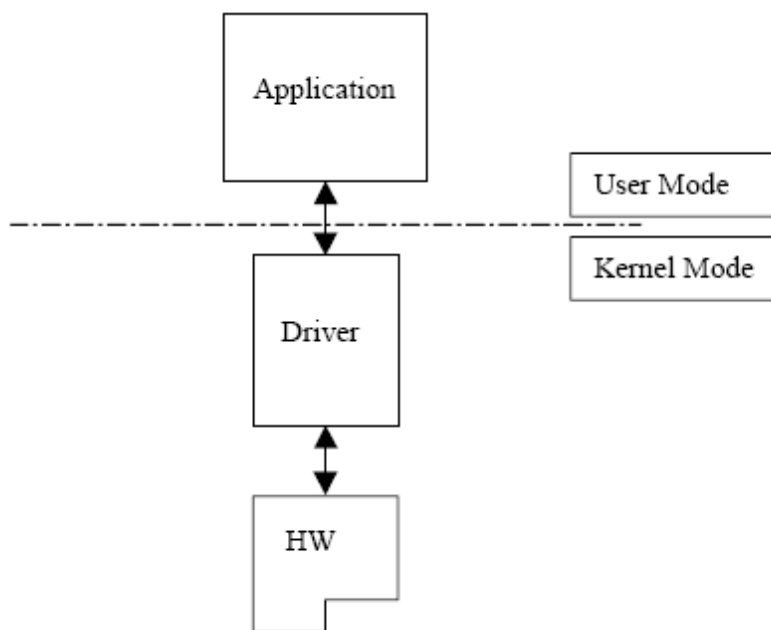
บทที่ 2

ทฤษฎีพื้นฐาน

2.1 ทฤษฎีในส่วนของ device driver

2.1.1 Device Driver Overview

ปัจจุบันคอมพิวเตอร์มีระบบปฏิบัติการหรือที่เรียกกันว่า Operating System (OS) อยู่มากมายไม่ว่าจะเป็น Windows , Linux , OS2 หรือ Solaris เป็นต้น ซึ่งในระบบปฏิบัติการต่างก็มีระบบการจัดการภายในระบบ ซึ่งการที่ OS นั้นจะทำการติดต่อกับ hardware นั้น จะไม่สามารถทำการติดต่อสื่อสารได้โดยตรง ดังนั้น จึงจะต้องมีการติดต่อผ่านกระบวนการที่เรียกว่า “Device Driver” ซึ่งจะเป็นสื่อกลางที่คอยติดต่อสื่อสารระหว่าง application กับ hardware



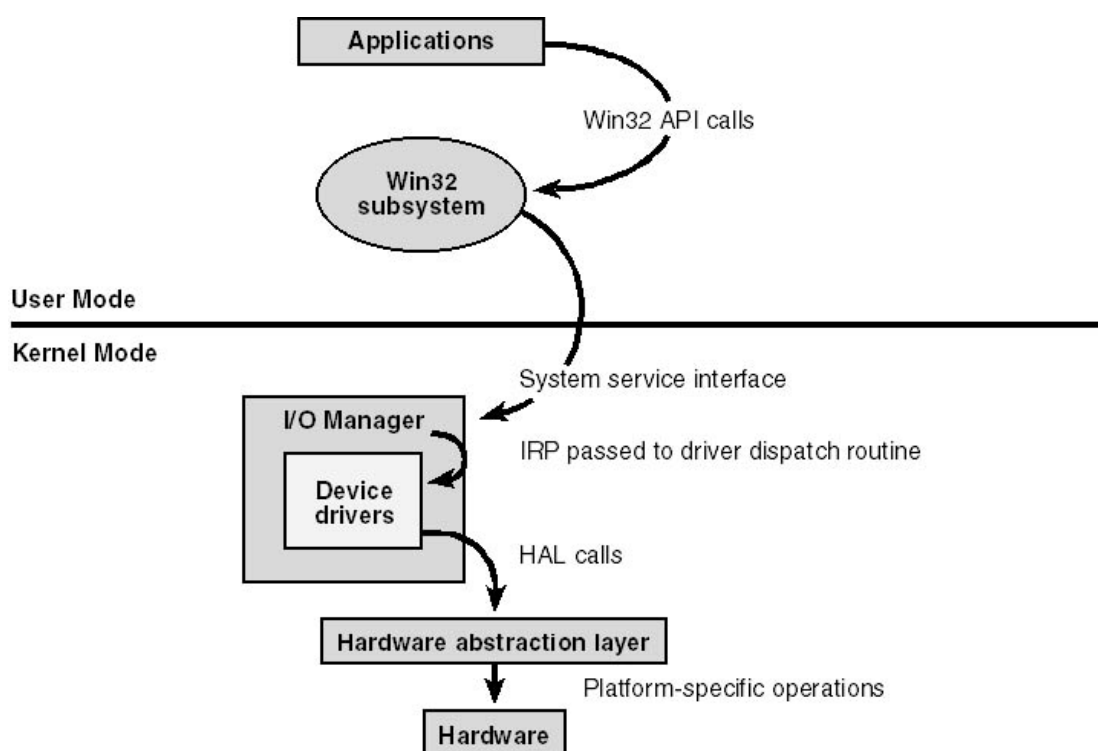
รูปที่ 2.1 แสดงความสัมพันธ์ของ Device Driver กับ ระบบ

จากรูปที่ 2.1 จะเห็นได้ว่ารูปถูกแบ่งออกเป็น 2 ส่วนคือ User Mode และ Kernel Mode ซึ่ง 2 ส่วนนี้จะมีหน้าที่การทำงานที่ต่างกัน ซึ่ง Application นั้นจะอยู่ในส่วนของ User Mode ซึ่งในส่วนนี้จะไม่สามารถที่จะติดต่อ เช่น อ่านหรือเขียนไฟล์ได้โดยตรง ทุกอย่างที่จะทำเกี่ยวกับ hardware นั้นจะต้องทำผ่าน Device Driver ซึ่งอยู่ในส่วนของ Kernel Mode ซึ่งทั้งสองส่วนนี้จะมีลำดับความสำคัญที่ไม่เท่ากัน คือในส่วนของ User Mode นั้นจะไม่สามารถที่จะติดต่อกับ hardware ได้โดยตรง ซึ่งต่างจากในส่วน of Kernel Mode ที่สามารถที่จะทำการติดต่อกับ hardware ได้โดยตรง

อย่างไรก็ตาม ในแต่ละระบบก็จะมี Layer ลำดับการทำงานที่แตกต่างกันออกไปแล้วแต่โครงสร้างของระบบนั้นที่ถูกออกแบบมาว่าจะมีการทำงานอย่างไร อย่างเช่นที่เราจะยกตัวอย่างเอามา 2 ตัวอย่างเพื่อให้เห็นถึงหลักการทำงานของระบบปฏิบัติการ ตัวอย่างที่เราจะนำมาใช้ในการเสนอก็คือ Windows 98 และ Windows 2000

2.1.2.Windows 2000

Windows 2000 Operating System Software ใดที่จะทำการ excute นั้น อย่างเช่น Application ที่ต้องการจะอ่านข้อมูลบางตัวจากอุปกรณ์ จะต้องไปเรียก Application Programming Interface (API) ไปเรียกที่ Subsystem Module เพื่อใช้ System service interface เพื่อที่จะไปติดต่อกับ I/O Manager



รูปที่ 2.2 แสดง Architecture ของ Windows 2000

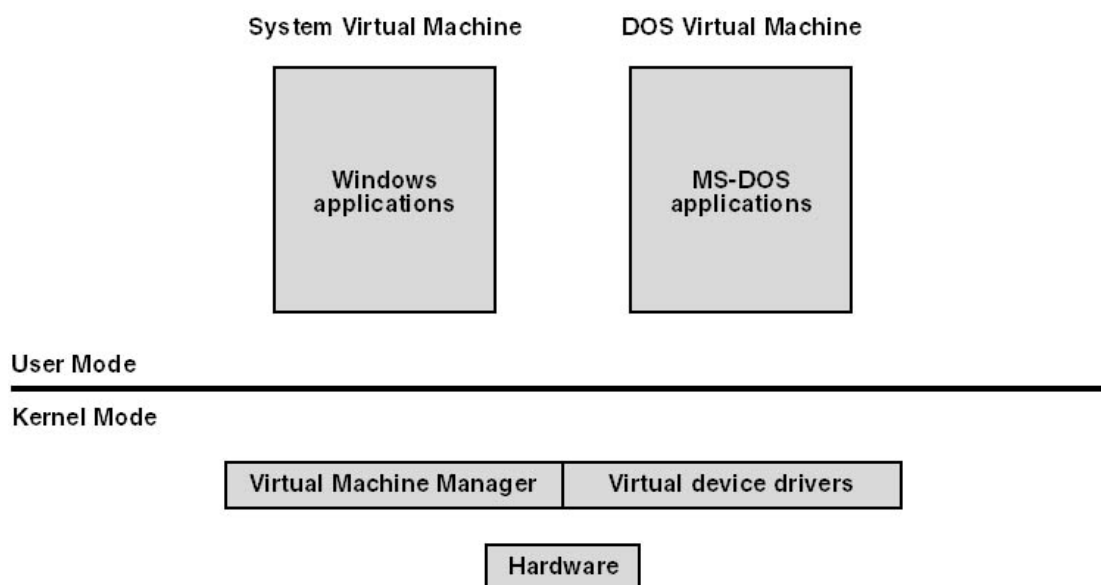
User Mode นั้นไม่สามารถ access ได้ด้วยตัวมันเอง พวกเขาจึงได้สร้าง data structure ที่เรียกว่า I/O request packet (IRP) เป็นตัวสื่อสารกับ Device Driver ซึ่ง Device Driver นั้นจะไปติดต่อผ่านทาง Hardware Abstraction Layer (HAL) เพื่อติดต่อกับ hardware อีกทีหนึ่ง ซึ่ง HAL นั้นจะเป็นตัวที่จะไปติดต่อกับ hardware จริงๆ ดังนั้นเราจะเห็นได้ว่าการทำงานของ Windows2000 นั้น การทำงานหลักๆ นั้น

จะทำงานอยู่ในส่วนของ I/O Manager ซึ่งจะเป็นคนคอยติดต่อสื่อสารกับ Device Driver ให้กับระบบปฏิบัติการ

2.1.3. Windows 98

รูปนี้แสดงให้เห็นถึงแนวคิดของ Windows 98 จากรูปที่3 จะพบว่ามันถูกแบ่งออกเป็น 2 ส่วนคือ ส่วนของ System Virtual Machine และ DOS Virtual Machine โดยที่ Kernel Mode จะเป็นตัวเรียก Virtual Machine Manager (VMM) เพราะว่า งานหลักของมันคือการสร้าง virtual machine ขึ้นมา Windows 98 ไม่สามารถจัดการ I/O ได้เหมือนกับ Windows2000ที่มีตัว I/O Managerคอยดูแลเรื่องของ I/O

ส่วนแตกต่างที่สำคัญของ Windows 98 ที่ต่างจาก windows2000 คือ การที่มีการจัดการติดต่อโดยตรงกับ disk หรือ port



รูปที่2.3 แสดงแนวคิดของ Windows98

2.1.4. Basic Structure ของ WDM Driver

ในส่วนของ basic structure ของ WDM Driver เราจะพูดถึงกันถึงใน 3 เรื่องหลักๆที่จำเป็นต้องใช้งานก็คือ

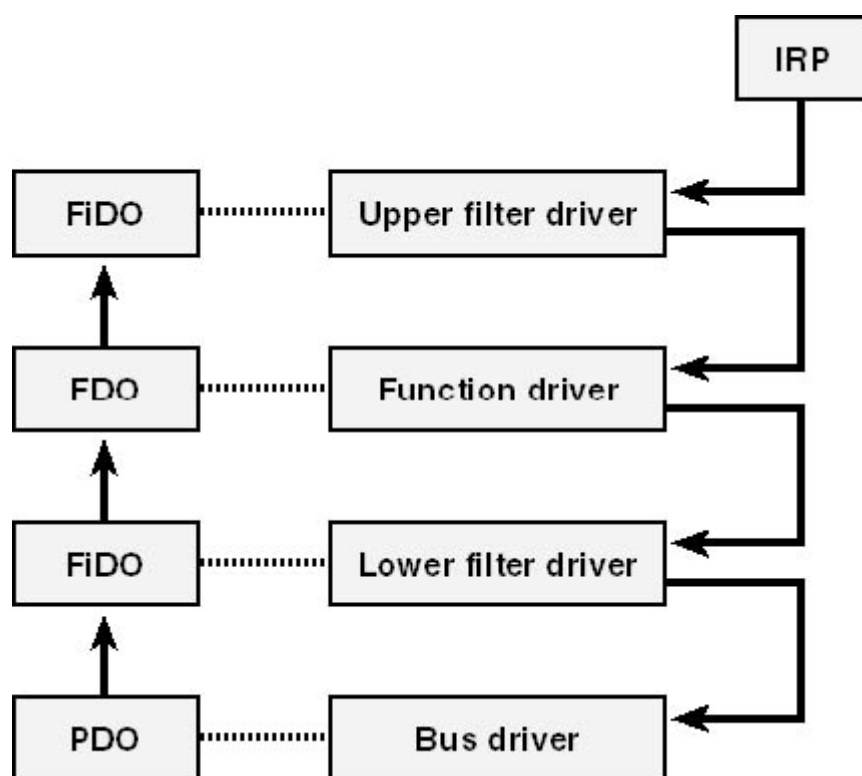
2.1.4.1. Device and Driver Layer

2.1.4.2. DriverEntry Routine

2.1.4.3. AddDevice Routine

ซึ่งทั้ง 3 ส่วนนี้จะเป็พื้นฐานเพื่อใช้ในการเขียน Device Driver ขึ้นมา ซึ่งเราจะต้องนำไปใช้งานต่อในเรื่องต่อไป

2.1.5. Device and Driver Layer



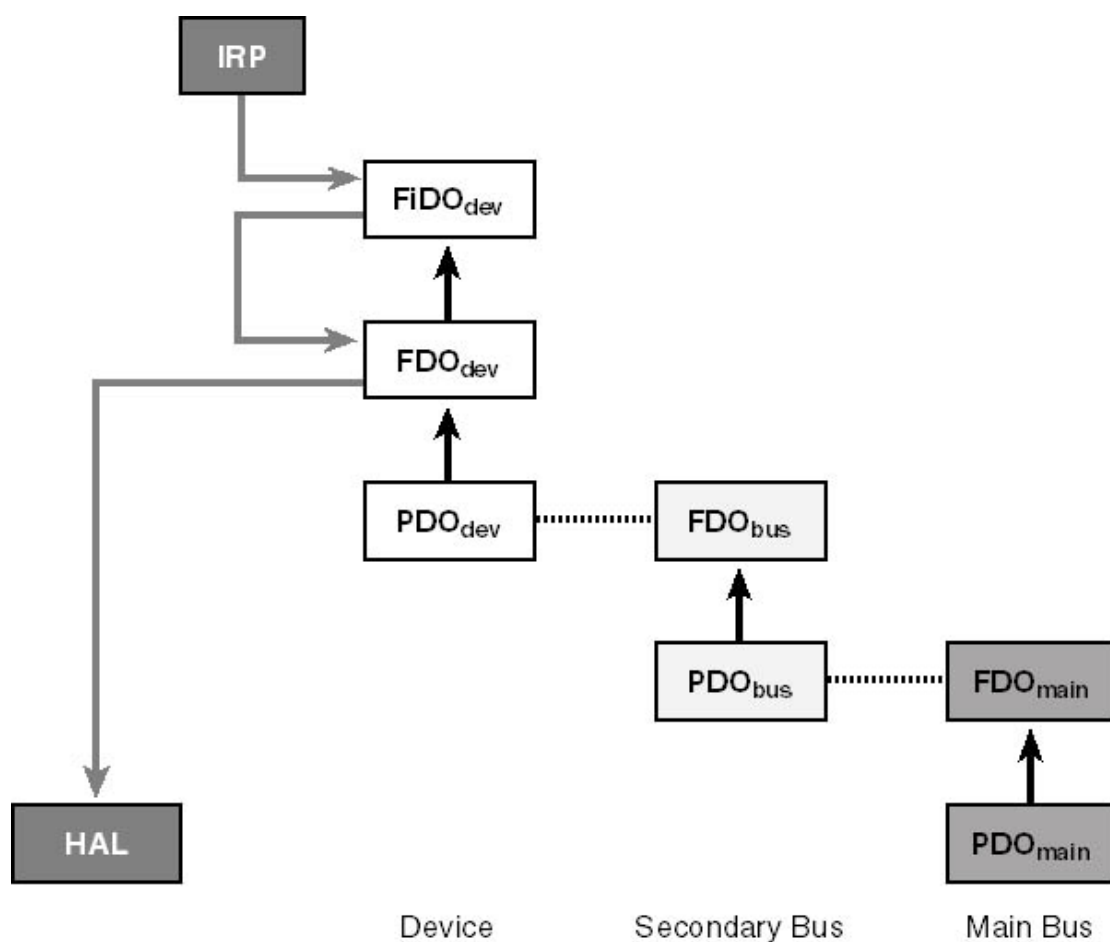
รูปที่ 2.4 แสดง layer ของ device object และ driver ใน WDM

เป็นรูปที่ 2.4 layer ของ Windows Driver Model of Driver Stack ของ device object เป็น data structure ซึ่ง System เป็นคนสร้างขึ้นมาเพื่อช่วยให้ software ที่จะไปจัดการกับ hardware ที่ชั้นล่างสุดของ device object ใน stack ที่เรียกว่า physical device object หรือ PDO ต่อมาเราจะมาดูในส่วนตรงกลางของ device object stack ซึ่งเป็นส่วนของ object ที่เรียกว่า function device object หรือ FDO ซึ่งส่วนที่อยู่ด้านบนหรือด้านล่างของ FDO จะสามารถเก็บส่วนของ filter device object ซึ่งอยู่เหนือ FDO เราจะเรียกว่า upper filter และด้วยเหตุนี้ filter device object ที่อยู่ด้านล่างของ FDO จึงถูกเรียกว่า lower filter

โดยที่ Plug and Play (PnP) Manager ของระบบปฏิบัติการจะเป็นคนสร้าง stack ของ device object การที่จะสร้าง PDO นั้น PnP Manager จะต้องพิจารณาที่ Registry database และ function driver ที่อยู่ตรงกลางของรูป จะมีการติดตั้งโปรแกรมเพื่อตอบสนองกับ Registry จำนวนมากที่จะเข้ามา โดยที่ Registry

Entry จะเป็นตัวที่คอยกำหนดว่า Driver ตัวไหนที่จะไปแสดงอยู่ใน stack ดังนั้นก่อนที่ PnP Manager จะไปโหลด Lowest filter driver และเรียก ฟังก์ชัน AddDevice มันก็จะสร้าง FiDO และสร้างเส้นทางเพื่อไปติดต่อระหว่าง FiDO และ Driver AddDevice ที่ต่อจาก PDO ไปยัง FiDO

วัตถุประสงค์ของการทำ layer จะปรากฏชัดเจนขึ้นเมื่อคุณพิจารณาการส่งผ่านของ IRP ซึ่งโดยปกติแล้ว IRP จะถูกส่งผ่านไปที่ส่วนบนสุดของ stack แล้วกรองผ่านลงมายัง stack อื่นต่อไป ซึ่ง driver จะเป็นตัวที่ตัดสินใจว่าจะทำอะไรกับ IRP บ้าง ซึ่งบางครั้งอาจจะไม่จำเป็นต้องทำไปถึงชั้นล่างสุดของ stack โดยที่ IRP อาจจะทำงานมาแล้วแค่ชั้นเดียวแล้วเสร็จเลยก็ได้ โดยสังเกตได้จากรูปที่ 2.2 ซึ่งจะแสดงการทำงานของ device object ที่มีการส่งผ่านค่า IRP ไปยัง FiDO เพื่อทำการกรองการทำงาน ซึ่งตัวอย่างนี้จะเป็นการที่อุปกรณ์ตัวหนึ่งต้องการที่จะติดต่อกับส่วนของ HAL จะพบว่าเมื่อมันส่ง IRP ไปที่ Upper filter แล้วทำการส่งต่อไปที่ FDO เพื่อทำการติดต่อกับ HAL โดยที่มันยังไม่ได้ติดต่อกับ PDO



รูปที่ 2.5 แสดงตัวอย่างการทำงานของ device object

2.1.6. DriverEntry Routine

ในส่วนก่อนหน้านี้ เราได้กล่าวไปว่า PnP Manager จะโหลดdriver ที่จำเป็นสำหรับhardware และเรียก AddDevice function ขึ้นมา ให้driverนั้นสามารถใช้กับhardwareได้และการทำglobal initial จะทำเพียงครั้งเดียวเท่านั้น เมื่อตอนที่โหลดขึ้นมาครั้งแรก ซึ่งglobal initial จะรับหน้าที่ในส่วนของ

DriverEntry routine

DriverEntry เป็นชื่อที่ถือเอาตามแบบธรรมดาที่ตั้งมาให้เป็นจุดที่เข้าไปสู่ Kernel Mode driver I/O Manager เรียก routineดังนี้

```
extern "C" NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
    IN PUNICODE_STRING RegistryPath)
{
    ...
}
```

ก่อนที่จะพูดถึงcodeที่อยู่ภายในDriverEntry เราต้องการพูดถึงถึงสิ่งสำคัญเล็กน้อยในเรื่องของfunction prototype ของตัวมันเอง เราจะใช้คำสัง extern “c” เพราะว่ามันเป็นกฎ เราจะใส่โค้ดของในรูปแบบของ C++

WDM ทำหน้าที่หลักๆในDriverEntry คือการเติมfunction pointต่างๆในdriver object pointer จะชี้ไปยังที่การทำงานที่พบsubroutine ซึ่งประกอบด้วยpointerที่เป็นสมาชิกของDriver Object

- DriverUnload
ต้อง set pointer นี้ไปที่ cleanup routine สร้าง I/O manager ควรจะเรียก routine นี้เมื่อต้องการ unload ไดรเวอร์ WDM driver ไม่ได้จาก resource ให้ใน DriverEntry ดังนั้น มันจึงไม่จำเป็นที่จะต้อง cleanup ทุกครั้ง
- DriverExtension->AddDevice
ชี้ pointer นี้ไปที่ AddDevice function ของคุณ PnP manager จะเรียก AddDevice ของแต่ละฮาร์ดแวร์ที่ตอบสนองคุณ ขณะที่ AddDevice มีความสำคัญในการทำงานของ WDM driver
- DriverStartIo
ถ้าไดรเวอร์คุณใช้ standard method ของ I/O request Queue คุณควรจะชี้ให้ member นี้ของ driver object ชี้ไปที่ StartIo routine ของคุณ
- MajorFunction
ตัว I/O manager มีการ initialize เวกเตอร์นี้ของฟังก์ชัน pointer to pointer to dispatch function ซึ่งมีการร้องขอล้มเหลวตลอด คุณควรจะเข้าไปในส่วนของคุณของ IRP ไดรเวอร์ของคุณ

```

extern "C" NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
    IN PUNICODE_STRING RegistryPath)
{
1 →  DriverObject->DriverUnload = DriverUnload;
    DriverObject->DriverExtension->AddDevice = AddDevice;
    DriverObject->DriverStartIo = StartIo;
2 →  DriverObject->MajorFunction[IRP_MJ_PNP] = DispatchPnp;
    DriverObject->MajorFunction[IRP_MJ_POWER] = DispatchPower;
    DriverObject->MajorFunction[IRP_MJ_SYSTEM_CONTROL] = DispatchWmi;
3 →  ...
4 →  servkey.Buffer = (PWSTR) ExAllocatePool(PagedPool,
    RegistryPath->Length + sizeof(WCHAR));
    if (!servkey.Buffer)
        return STATUS_INSUFFICIENT_RESOURCES;
    servkey.MaximumLength = RegistryPath->Length + sizeof(WCHAR);
    RtlCopyUnicodeString(&servkey, RegistryPath);
5 →  return STATUS_SUCCESS;
}

```

1. จะเห็นว่าทั้ง 3 statement จะ set function pointer ให้กับ entry pointer ใน driver โดยเราได้เลือกเอาชื่อฟังก์ชันง่ายๆ คือ DriverUnload , AddDevice และ StartIo
 2. WDM driver ทั่วๆ ตัว ต้องมีการจัดการ PNP, POWER และ SYSTEM_CONTROL I/O request
 3. ในส่วนนี้ เราละไว้ในฐานที่เข้าใจ มันคือส่วนที่เราต้องมาเขียนเสริมต่อเอาเอง
 4. ถ้าคุณจำเป็นต้องติดต่อกับ service registry key อื่นใน Driver ของคุณ มันเป็นความคิดที่ดีที่ copy เอา RegistryPath ถ้าคุณจะไปทำงานที่ WMI (Windows Management Instrumentation) ซึ่งในตัวอย่าง เราสมมุติว่าคุณประกาศ global variable ที่มีชื่อว่า servkey เป็น UNICODE_STRING
 5. การ return STATUS_SUCCESS เป็นวิธีที่คุณแสดงว่ามันทำเสร็จแล้ว ถ้าคุณตรวจพบว่ามีผิดพลาด มันจะ return error code จาก NTSTATUS.H หรือจาก set ของ error code ซึ่งคุณเป็นคนกำหนดเอง
- DriverUnload

วัตถุประสงค์ของ WDM driver ที่ เป็น DriverUnload function เป็นการ clean up หลังจาก global initialization ใดๆ ซึ่ง DriverEntry สามารถทำได้

2.1.7. AddDevice Routine

ในส่วนของก่อนหน้านี้ เราได้แสดงให้เห็นถึงวิธีการ initialize ที่ WDM driver เมื่อครั้งที่เริ่มทำการโหลดครั้งแรก โดยทั่วไปนั้น driver นั้นสามารถเรียกหรือจัดการ device มากกว่าหนึ่ง device ใน WDM architecture driver มีฟังก์ชันพิเศษชื่อว่า AddDevice ซึ่ง PnP Manager สามารถเรียกใช้แต่ละ device ได้ ซึ่ง function มี prototype ดังนี้

```
NTSTATUS AddDevice(PDRIVER_OBJECT DriverObject, PDEVICE_OBJECT pdo)
{
}
```

driver objectนี้เป็นจุดเดียวกับdriver objectที่ถูกinitialในDriverEntry routine ของคุณ ซึ่ง PDO คือ addressของphysical device object ที่อยู่ด้านล่างของstack

พื้นฐานการตอบสนองของAddDeviceในfunction driver เป็นการสร้างdevice objectและlinkมันไปสู่ stack root ใน PDO ซึ่งมีขั้นตอนดังนี้

2.1.7.1. เริ่มโดยการเรียก IoCreateDeviceในการสร้างdevice objectและกรณีของdevice objectที่เป็นส่วนเพิ่มเติมมาของคุณ

2.1.7.2. Applicationรู้ว่ามีการRegister ที่ตัวที่อยู่ในdeviceของคุณ ในทางกลับกันเราให้device object nameและสร้างsymbolic link

2.1.7.3. ต่อมาให้initialize deviceที่เพิ่มเติมของคุณและflag memberของdevice object

2.1.7.4. เรียก IoAttachDeviceToDeviceStackวางลงในdevice objectตัวใหม่ไปสู่ stack

ดังนั้นเราจะมาดูในรายละเอียดขั้นตอนของ Creating a Device Object ซึ่งคุณสร้าง device object โดยการเรียก IoCreateDevice ดังตัวอย่าง

```
PDEVICE_OBJECT fdo;
NTSTATUS status = IoCreateDevice(DriverObject,
    sizeof(DEVICE_EXTENSION), NULL,
    FILE_DEVICE_UNKNOWN, FILE_DEVICE_SECURE_OPEN, FALSE, &fdo);
```

ในArgumentแรกเราจะมาดูกันที่ Driverobject จะเป็นตัวเดียวกับAddDevice ซึ่งDriverobject จะสร้างการติดต่อระหว่างdriverของคุณและ device objectตัวใหม่ ดังนั้น มันจะยอมให้I/OManagerส่ง IRPไปให้device

ในส่วนต่อมาargumentนี้เป็นขนาดของdeviceที่ถูกสืบทอดมา ในส่วนนี้เราได้อธิบายอย่างง่ายซึ่งในส่วนของI/O Manager ได้กำหนดการเพิ่มmemoryและset DeviceExtension pointer ใน device object ไปที่pointerของมัน

Argumentที่สามซึ่งเป็น NULLในตัวอย่างสามารถเป็นaddressของ UNICODE_STRING จัดไว้ให้device object ตัดสินใจ

Argumentที่สี่(FILE_DEVICE_UNKNOWN) เป็นส่วนหนึ่งของ device type ซึ่งค่าอะไรก็ตามของคุณกำหนดที่นี่ คุณสามารถoverride โดยเข้าไปในdeviceที่เป็นhardware key หรือclass key ถ้ามีทั้งคู่ overrideเราจะให้ hardware key มีอิทธิพลที่มากกว่า

Argument ที่ห้า(0) จัดเตรียมCharacteristic flag ไว้ให้ device object ซึ่งflagเหล่านี้มีความสัมพันธ์กับ mass storage device ซึ่ง FILE_AUTOGENERATED_DEVICE_NAME จะไม่มีเอกสารอ้างอิง จะมีอยู่แต่ในการใช้ภายในเท่านั้น เพราะ เอกสารของ DDK ลืมพูดถึงมัน อย่างไรก็ตาม value ที่คุณกำหนดที่นี่สามารถ overridden โดยเข้าไปใน device hardware key หรือ class key

Argument ที่หก IoCreateDevice (ในตัวอย่างเป็น FALSE) แสดงให้ว่า device เป็น exclusive ซึ่ง I/O Manager จะยอมให้เข้าไปจัดการได้เพียงตัวเดียวเท่านั้น โดย exclusive device อย่างไรก็ตาม value ที่คุณกำหนดที่นี่สามารถ overridden โดยเข้าไปใน device hardware key หรือ class key

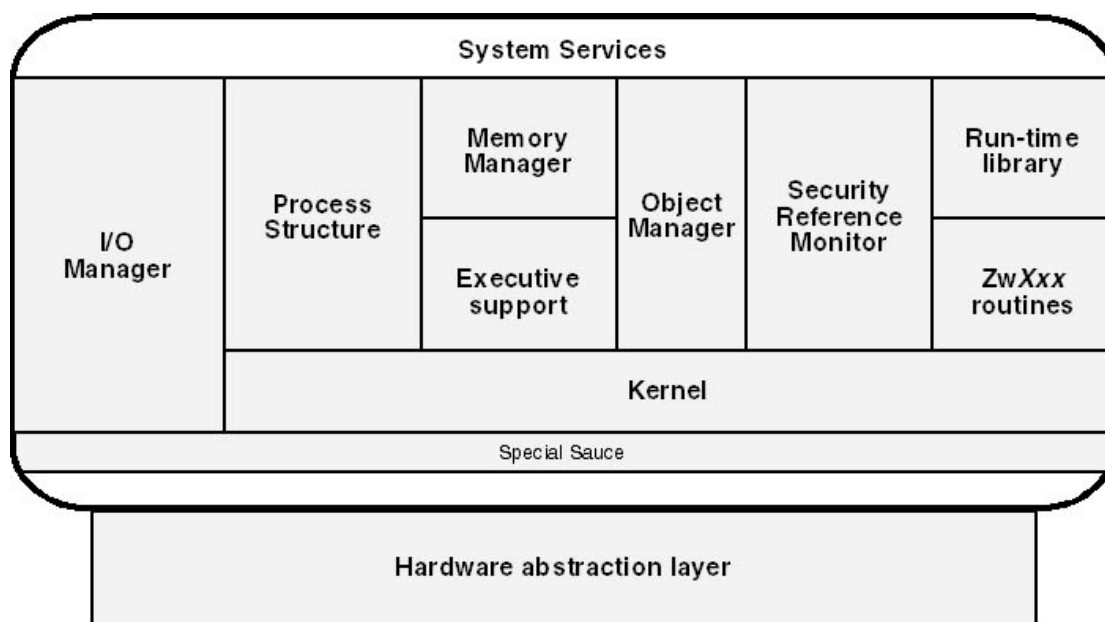
Argument สุดท้าย (&fdo) เป็นตำแหน่งที่ IoCreateDevice จะเก็บ address ของ device object ที่มันสร้าง

ถ้า IoCreateDevice ล้มเหลวในบางสาเหตุ มันจะ return ค่า status code และไม่ทำการเปลี่ยนแปลง PDEVICE_OBJECT แต่ถ้ามัน succeed มันจะ return ค่า successful ของ status code และ set ค่า PDEVICE_OBJECT คุณสามารถกระทำต่อไปโดยการ initialize device ของคุณที่สืบทอดมาและทำงานอย่างอื่นที่เกี่ยวข้องกับการสร้าง device object ตัวใหม่ คุณควรจะค้นพบ error หลังจากจุดนี้ไปและคุณควรจะแก้ไข device object และ return status code

```
NTSTATUS status = IoCreateDevice(...);
if (!NT_SUCCESS(status))
    return status;
...
if (<some other error discovered>)
{
    IoDeleteDevice(fdo);
    return status;
}
```

2.2 Basic Program

2.2.1. Basic Programming techniques



รูปที่ 2.6 แสดง kernel support routine

ในส่วนนี้เราจะมาพูดถึงเรื่องทั่วไปของการเขียน kernel mode ดังรูปที่ 2.6 แสดงบางส่วนของ Microsoft Windows NT operating system ซึ่งในแต่ละส่วนเป็น function การบริการซึ่งมีชื่อนำหน้าอยู่ 2 ตัว ดังนี้

- **I/O Manager (Io)** เป็นตัวที่เก็บ function จำนวนมากซึ่ง driver เป็นคนเรียกใช้
- **Process Structure module (Ps)** เป็นตัวสร้างและจัดการ kernel-mode thread ซึ่งโดยปกติแล้ว WDM driver สามารถจะใช้ thread ได้อย่างอิสระ
- **Memory Manager (Mm)** เป็นตัวควบคุม page table ซึ่งกำหนด mapping ของ virtual address ไปสู่ physical memory
- **Executive (Ex)** เป็นส่วนของ heap management และ synchronization
- **Object Manager (Ob)** เป็นศูนย์กลางควบคุม data object จำนวนมากในการทำงานของ Window NT ซึ่ง WDM driver ใ้ใจ Object Manager เท่านั้นสำหรับที่จะเก็บจำนวนที่ไว้อ้างอิง เพื่อป้องกัน object จากการสูญหาย
- **Security reference Monitor (Se)** จะยอมให้ทำการ check ที่ File system driver หรือไม่ บางครั้งในเรื่องของความปลอดภัยในเวลา I/O request ไปที่ WDM driver

- **Run-time library (Rtl)** เป็นส่วนที่เก็บutility เช่น list และ string manager routine ซึ่งkernel mode สามารถใช้แทน ANSI standard library ส่วนใหญ่การทำงานในส่วนนี้จะดูได้จากชื่อของมัน
- Windows NT implement user mode ไปเรียก Win32 subsystemในkernel mode เกี่ยวกับ routine ที่มีชื่อนำหน้าว่า Zw ซึ่งประกอบไปด้วยfunctionที่เอาไว้สำหรับ access file และ registry
- **Windows NT kernel (Ke)** อยู่ทุกที่ในlow – level synchronizationของ activities ระหว่าง threads และ processors
- Layerล่างสุดของoperating system อยู่ที่**hardware abstraction layer (HAL)** HAL รู้ถึงวิธีการทำงานของinterrupt ,spinlock,address I/O และmemory map device ดังนั้นแทนที่จะพูดคุยโดยตรงกับhardwareนั้น WDM driverจะไปเรียกfunctionในHALขึ้นมาทำแทน

2.2.2.Side Effect

เราจะมาคุยกันถึงในเรื่องของ Side Effect ว่ามีผลยังไงต่อโปรแกรมที่เราจะเขียนซึ่งบางคนอาจจะยังไม่รู้ถึงผลกระทบที่จะได้ในการประกาศหรือเขียนโปรแกรม ดังตัวอย่างที่อยู่ด้านล่างนี้

```
int a = 2, b = 42, c;
c = min(a++, b);
```

จากโปรแกรมด้านบนค่าของ a และ c จะมีค่าเท่าไร

```
#define min(x,y) ((x) < (y)) ? (x) : (y)
```

ถ้าคุณเอา a++มาแทน x คุณจะเห็นว่า a มีค่าเท่ากับ 4 เพราะ a++ได้รับการทำไป2ครั้ง แต่ค่าminจะเท่ากับ 3 ซึ่งคุณควรจะทำตามกฎเมื่อคุณเขียนโปรแกรม WDM driver

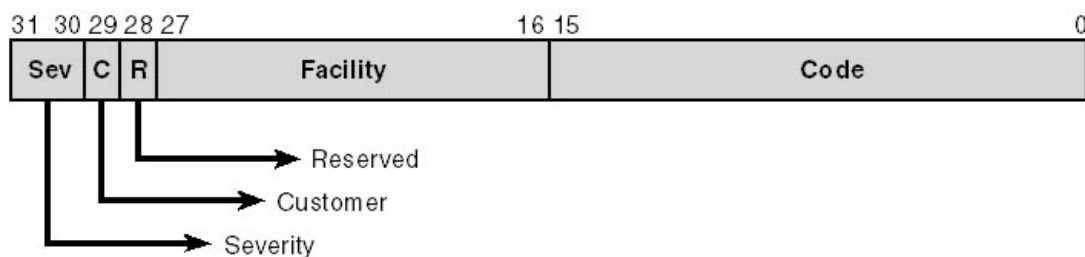
2.2.3.Error Handling

ความผิดพลาดส่วนใหญ่จะเกิดขึ้นจากมนุษย์เป็นคนทำ ซึ่งการทำงานในOperating System นั้นห้ามเกิดความผิดพลาด เพราะถ้าเกิดความผิดพลาดจะทำให้ระบบนั้นล่มหรือเกิดความเสียหายในส่วนต่างๆ ได้

ดังนั้นเราจึงทำการป้องกันความผิดพลาด โดยทั่วไป kernel mode สนับสนุน routineที่มีการ return status code ในการที่ตรวจจับความผิดพลาด โดยการรายงานความผิดพลาดนั้นอาจจะอยู่ในรูปแบบของ Boolean หรือ numerice value

2.2.4. Status code

Kernel mode มีการแสดงว่า success หรือ failure โดยการ return status code ไปที่ผู้เรียก ซึ่ง NTSTATUS value เป็น integer ขนาด 32 bit ประกอบด้วย subfield ต่างๆดังรูปที่ 3.5 บิตที่อยู่สูงสุด 2 บิต ใช้แสดงว่า report นั้น success, information, warning หรือ error ต่อมาเรามาดูที่ customer bit ใช้สำหรับการเปลี่ยนแปลงหรือแก้ไขของเครื่อง mainframe operating system ซึ่งเราไม่คิดว่าปัจจุบันจะยังได้ใช้อยู่ ต่อมาเรามาดูที่ facility code แสดงให้เห็นถึง system component ดังนั้นส่วนที่เหลือคือส่วนของ status code อีก 16 บิต แสดงเงื่อนไขของ report



รูปที่ 2.7 แสดงของ NTSTATUS code

คุณควรเช็ค status ที่ return มาจาก routine นั้น จะไปหยุดหน้าที่การทำงานที่ทำอยู่เป็นประจำของ code นั้น ถ้า high bit ของ status code เป็น 0 แล้วส่วนที่เหลือเป็นจำนวนใดๆแสดงว่า success

```
NTSTATUS status = SomeFunction(...);
if (!NT_SUCCESS(status))
{
    <handle error>
}
```

ตัวอย่าง การขั้นตอน initial AddDevice function กับ error checking

```

NTSTATUS AddDevice(PDRIVER_OBJECT DriverObject, PDEVICE_OBJECT pdo)
{
    NTSTATUS status;
    PDEVICE_OBJECT fdo;
    status = IoCreateDevice(DriverObject, sizeof(DEVICE_EXTENSION),
        NULL, FILE_DEVICE_UNKNOWN, 0, FALSE, &fdo);
1 → if (!NT_SUCCESS(status))
    {
2 →     KdPrint(("IoCreateDevice failed - %X\n", status));
        return status;
    }
    PDEVICE_EXTENSION pdx = (PDEVICE_EXTENSION) fdo->DeviceExtension;
    pdx->DeviceObject = fdo;
    pdx->Pdo = pdo;
    pdx->state = STOPPED;
3 → IoInitializeRemoveLock(&pdx->RemoveLock, 0, 0, 255);
    status = IoRegisterDeviceInterface(pdo, &GUID_SIMPLE, NULL,
        &pdx->ifname);
4 → if (!NT_SUCCESS(status))
    {
        KdPrint(("IoRegisterDeviceInterface failed - %X\n", status));
        IoDeleteDevice(fdo);
        return status;
    }
    ...
}

```

1. ถ้า IoCreateDevice fails เราจะreturn status code กลับไปให้มัน
2. เป็นการdebugging ตัวdriverโดยให้มันแสดง errorที่มันพบ
3. IoInitializeRemoveLock เป็นVOID function ซึ่งหมายความว่ามันจะไม่สามารถที่otfailได้ ดังนั้นเราไม่จำเป็นต้องcheck status code
4. ถ้า IoRegisterDeviceInterface เกิด fail เราจะมีcleanup ก่อนที่จะreturnไปให้ผู้เรียกมัน โดยที่คุณจะต้องเรียกใช้ IoDeleteDevice ให้การทำลายdevice object ที่เพิ่งสร้างมา

2.2.5. Memory Management

ในส่วนนี้จะอธิบายถึงmemory management ซึ่ง windows2000มีการแบ่ง virtual address space อยู่หลากหลายวิธี

windows2000มีวิธีใช้ memory management อยู่หลายวิธี โดยเราจะพูดถึง2วิธีพื้นฐานคือ ExAllocatePool และ ExFreePool ให้คุณไว้ใช้สำหรับ allocate และ release ขนาดของblockจาก heap เราจะอธิบายอย่างคร่าวๆว่าคุณจะใช้ memory blockไปไว้ในlink listsของ structure สุดท้ายเราจะอธิบาย concept ของ lookaside list ที่ซึ่งอนุญาตให้คุณให้ allocate และ release block นั้น

How Big Is a Page?

ใน virtual memory system , physical memory และ swap file ไปสู่ page frame ใน WDM driver คุณสามารถใช้ PAGE_SIZE บอกว่าคุณมีpageขนาดเท่าไร ซึ่งใน Windows NTนั้น page มีขนาด 4096 byte

```
PAGE_SIZE == 1 << PAGE_SHIFT
```

คุณสามารถใช้macrosในการทำงานเกี่ยวกับขนาดของpage

-ROUND_TO_PAGES

-BYTES_TO_PAGES

-BYTE_OFFSET

-PAGE_ALIGN

-ADDRESS_AND_SIZE_TO_SPAN_PAGES

2.2.6.Synchronization

Microsoft Windows 2000เป็น multitasking operation system ซึ่งสามารถทำงานใน symmetric multiprocessor environment ซึ่งเราจำเป็นต้องรู้และเข้าใจผู้เขียนdriverซึ่งจะต้องนำcodeไปexecute ในcontext ของ threadๆหนึ่งและในภาวะฉุกเฉินของ multitasking สามารถควบคุมจากการทำงานของmomentใดๆ ยิ่งกว่านั้นการที่executionของmultiple thread เป็นกลไกบนmultiprocessor ซึ่งwindowsอนุญาตให้คุณแก้ไขปัญหามาโดยการใช้interrupt request level (IRQL) priority scheme และการอ้างถึง และ แก่spin lock ซึ่งcodeในส่วนนี้ IRQL หลีกเลี่ยงการทำลายpreemptionบนsingle cpu

2.2.7.Interrupt Request Level

Windows NT มีการกำหนด priority level ที่รู้จักกันในชื่อ interrupt requestซึ่ง IRQL จะจัดเตรียมวิธีการ synchronization สำหรับการทำงานบน single CPU โดยมีกฎดังนี้:

CPU หนึ่งจะรัน IRQL บน PASSIVE_LEVEL ได้ การทำงานบน CPU สามารถ preempt ได้โดยการทำงาน execute ของตัวที่มี IRQL สูงกว่า

รูปที่2.8 จะแสดงการจัดลำดับค่าของ IRQL สำหรับ x86 platform โปรแกรมที่รันบน User-mode จะรัน PASSIVE_LEVEL เพราะฉะนั้นการ preempt จะทำการ execute และเลื่อนลำดับ IRQL ให้สูงขึ้น มีฟังก์ชันมากมายที่ถูก execute ที่ PASSIVE_LEVEL โดย DriverEntry และ AddDevice ได้อธิบายไว้ในบทที่ 2 เรื่อง “Basic Structure of a WDM Driver” ส่วนในบทนี้จะเป็น I/O request packet (IRP) เสียส่วนใหญ่

Driver routine พื้นฐานจะ execute ที่ Dispatch_Level ซึ่งสูงกว่า PASSIVE_LEVEL จะมีการรวม StartIo, deferred procedure call (DPC) routines และฟังก์ชันอื่นๆ เข้าไปด้วย ในส่วนของพื้นฐานที่ต้องการติดต่อพื้นที่ส่วนของ device object และ device extension ที่ปราศจากการแทรกแซงจาก driver dispatch routine เมื่อ routines นี้กำลังรันอยู่จะรับประกันว่าจะไม่มี thread ที่สามารถ preempt การ execute จาก driver dispatch routine ได้เพราะ dispatch routine รันที่ IRQL ที่ต่ำกว่า นอกจากนี้ไม่มี thread ที่สามารถ preempt ได้เพราะรันอยู่ใน IRQL เดียวกัน การที่จะ preempt ได้จะต้องถูก preempt จาก thread ที่มี IRQL สูงกว่าเท่านั้น

HIGH_LEVEL	31
POWER_LEVEL	30
IPI_LEVEL	29
CLOCK2_LEVEL	28
CLOCK1_LEVEL	28
PROFILE_LEVEL	27
...	
DISPATCH_LEVEL	2
APC_LEVEL	1
PASSIVE_LEVEL	0

รูปที่ 2.8 Interrupt request level

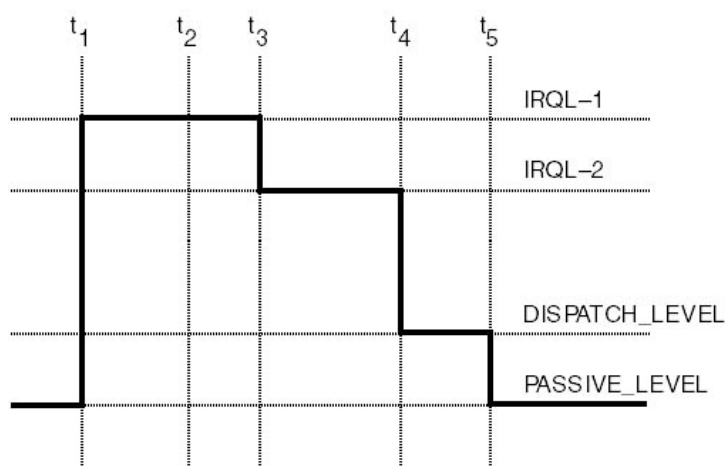
ระหว่าง DISPATCH_LEVEL และ PROFILE_LEVEL คือห้องสำหรับ interrupt level ของอุปกรณ์ ภายนอกต่างๆ แต่ละอุปกรณ์จะมีการสร้าง interrupt ที่มี IRQL ที่มีกำหนดลำดับความสำคัญเมื่อมาเปรียบเทียบกับอุปกรณ์อื่น WDM มีการจัดการ IRQL สำหรับ interrupt ของมันเมื่อมันได้รับ

IRP_MJ_PNP request ด้วยฟังก์ชัน minor code IRP_MN_START_DEVICE. Interrupt level ของอุปกรณ์มีการกำหนดข่าวสารสำหรับส่งไปให้ตัวแปรของการร่อนนั้นๆ เราจะอ้างถึงอุปกรณ์ IRQL โดยเรียกสั้นๆว่า DIRQL ซึ่งมันไม่ใช่ single request level ในทางตรงกันข้ามมันเป็น IRQL สำหรับการร่อนขอเชื่อมความสัมพันธ์กับอุปกรณ์ต่างๆ ซึ่งจะอธิบายในหัวข้อต่อไป

IRQL level อื่นๆ มีจุดประสงค์บางอย่างแล้วแต่สถาปัตยกรรม CPU นั้นๆ ตั้งแต่ level ถูกนำมาใช้ใน Windows NT kernel มันไม่มีจุดประสงค์พิเศษในการเชื่อมความสัมพันธ์งานการเขียน device driver จุดประสงค์ของ APC_LEVEL สำหรับตัวอย่างคือยอมให้ระบบกำหนด asynchronous procedure call (APC) ซึ่งจะอธิบายรายละเอียดที่ตอนท้ายของบทนี้ สำหรับ thread จะปราศจากการแทรกแซงจาก thread อื่นๆภายใน CPU เดียวกัน การทำงานจะเกิดขึ้นที่ HIGH_LEVEL

2.2.8. IRQL in Operation

จะแสดงให้เห็นถึงความสำคัญของ IRQL ซึ่งแสดงในรูปที่ 2.9 ซึ่งแสดงให้เห็นถึงความเป็นไปได้ของเหตุการณ์บน Single CPU เมื่อการเริ่มต้นของลำดับ CPU จะทำการ execute ที่ PASSIVE_LEVEL ที่เวลา t_1 การขัดจังหวะมาถึงบริการ routine execute ที่เวลา IRQL-1 ระดับหนึ่งระหว่าง DISPATCH_LEVEL และ PROFILE_LEVEL ที่เวลา t_2 มาถึงการขัดจังหวะของบริการ routine execute ที่ IRQL-2 ซึ่งน้อยกว่า IRQL-1 เพราะกฎการ preempt แสดงถึงการบริการต่อเนื่องของ CPU เมื่อการขัดจังหวะครั้งแรกของบริการ routine เสร็จสิ้นที่เวลา t_3 มันอาจจะส่ง request DPC ซึ่ง DPC routine จะ execute ที่ DISPATCH_LEVEL ผลที่ตามมาเมื่อตัวที่มีลำดับความสำคัญสูงกว่ากำลังทำงานบริการ routine เพื่อการขัดจังหวะครั้งที่ 2 เพราะฉะนั้นการ execute ถัดไป เมื่อเวลา t_4 เสร็จสิ้น การกำหนดไม่สามารถทำให้มีอะไรเกิดขึ้นในเวลาเฉลี่ย DPC จะรันที่ DISPATCH_LEVEL เมื่อ DPC routine เสร็จสิ้นที่เวลา t_5 IRQL จะสามารถหยุดอยู่ที่ PASSIVE_LEVEL ได้



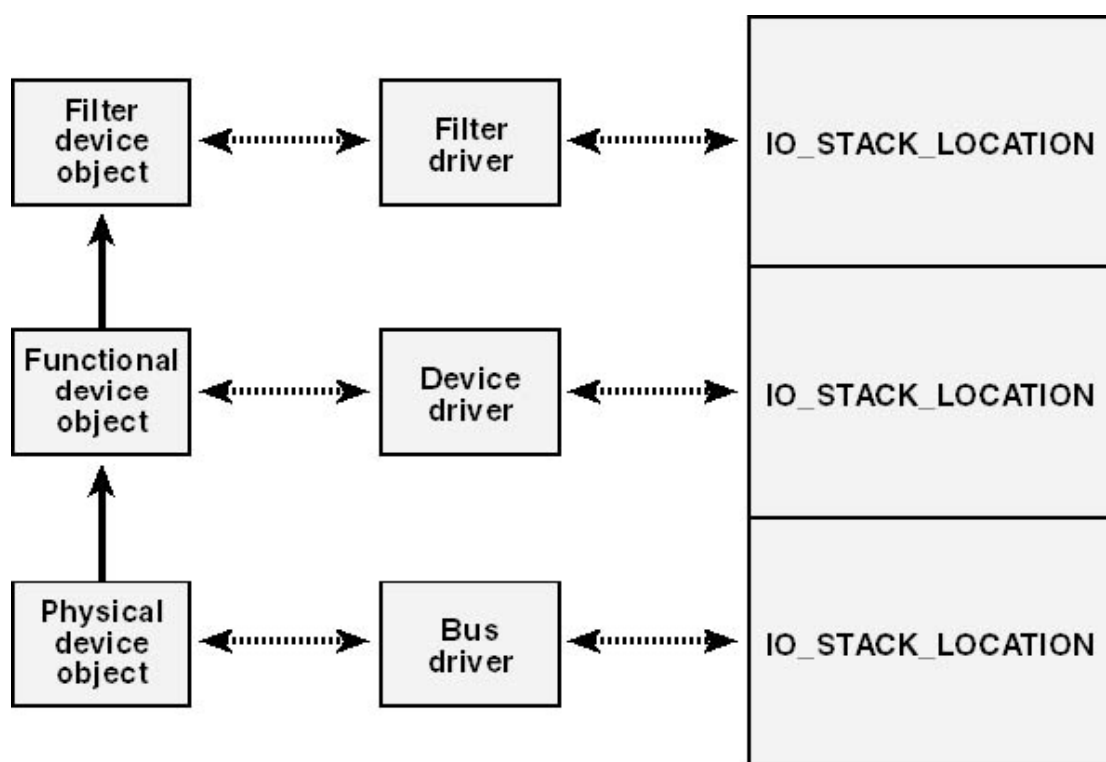
รูปที่ 2.9 Interrupt priority

2.2.9.The I/O Request Packet

Operating system ใช้data structure ที่เรียกว่า I/O request packet หรือ IRP ในการติดต่อกับ kernel-mode ในบทนี้ได้อธิบายเกี่ยวกับส่วนที่สำคัญของdata structure และโดยหมายถึง create , sent ,process และ ultimately destroy สุดท้ายเราจะอธิบายถึงความสัมพันธ์ของส่วนประกอบต่างๆของแนวคิดซึ่งสิ่งแวดล้อมจะเป็นตัวกำหนดชนิดของIRP เพราะฉะนั้นคุณอาจต้องการความรู้ในบทนี้และอ้างอิงที่ซึ่งคุณกำลังอ่านในบทนี้

2.2.10.The I/O Stack

เมื่อไหร่ก็ตามที่ทุกๆkernel-mode สร้างIRP มันจะสร้างความสัมพันธ์ด้วยarrayของโครงสร้าง IO_STACK_LOCATIONโดยที่ตำแหน่งของstackตำแหน่งหนึ่งสำหรับแต่ละdriverซึ่งprocessเป็น IRPและตำแหน่งของstackเดี่ยวๆเป็นประจำสำหรับใช้ของIRPตัวเดิม(ดูรูปที่ 2.10) ตำแหน่งstackเก็บชนิดของcode และ parameter สำหรับIRP ด้วยaddressของroutineที่ทำเสร็จแล้ว



รูปที่2.10 parallelismระหว่างdriverและI/O stack

2.2.11. The DeviceIoControl API

เป็นขั้นตอนการใช้งานฟังก์ชันมาตรฐานต่างๆ ของ Microsoft Win32 ซึ่งเป็นฟังก์ชันที่ทำงานบน user-mode driver การทำงานของฟังก์ชันจะทำการติดต่อกับอุปกรณ์ฮาร์ดแวร์ ผ่าน kernel-mode driver โดยจะส่ง IRP ไปบอกให้ kernel-mode driver ทราบว่าตอนนี้ user-mode driver ต้องการทำอะไร รายละเอียดของฟังก์ชันต่างๆ มีดังนี้

2.2.11.1 ฟังก์ชัน DeviceIoControl()

เป็นการเรียกใช้ฟังก์ชันมาตรฐาน ของ Microsoft Win32 ซึ่งเป็นฟังก์ชันที่ทำงานบน user-mode driver และการทำงานจะส่ง IRP_MJ_DEVICE_CONTROL ไปให้กับ kernel-mode driver เพื่อทำการติดต่อกับอุปกรณ์ฮาร์ดแวร์ โดยฟังก์ชันที่ใช้ในการทำงานติดต่อกับอุปกรณ์ฮาร์ดแวร์นี้คือฟังก์ชัน DeviceIoControl มีรูปแบบการใช้งานดังนี้

```
result = DeviceIoControl(Handle, Code, InputData, InputLength,
    OutputData, OutputLength, &Feedback, &Overlapped);
```

ค่าที่ได้รับจากการเรียกใช้ฟังก์ชันนี้คือ ค่า Boolean ถ้าสามารถติดต่อกับ kernel-mode driver หรือว่าภายในระบบปฏิบัติการวินโดวส์มี device driver ที่สามารถตอบสนองต่อ IOControl Code นี้ได้ก็จะส่งค่ากลับเป็น TRUE ถ้าไม่สามารถติดต่อกับ kernel-mode driver หรือว่าภายในระบบปฏิบัติการวินโดวส์ไม่มี device driver ที่สามารถตอบสนองต่อ IOControl Code นี้ได้ก็จะส่งค่ากลับเป็น FALSE และค่า parameter ที่ส่งไปมีทั้งหมด 8 ตัว แต่มีตัวหลัก ๆ เพียง 6 ตัวคือ

ค่า parameter ตัวที่ 1 เป็นค่า Handle ของอุปกรณ์ฮาร์ดแวร์ที่เราต้องการจะติดต่อกับ ได้รับมาจากการใช้ฟังก์ชัน CreateFile ซึ่งจะอธิบายในหัวข้อถัดไป

ค่า parameter ตัวที่ 2 เป็นค่า IOControl Code ที่โปรแกรม user-mode driver ส่งไปให้กับ kernel-mode driver ซึ่งค่า IOControl Code นี้จะมีการประกาศค่าไว้ใน kernel-mode driver และโปรแกรม user-mode driver ที่ต้องการจะติดต่อกับ ต้องส่งค่า IOControl Code ที่ตรงกับที่ได้ประกาศไว้ใน kernel-mode driver ด้วย ถ้าหากมีการส่งค่าที่ไม่ตรงกับที่ประกาศไว้ใน kernel-mode driver ก็จะไม่สามารถติดต่อกับอุปกรณ์ฮาร์ดแวร์ได้ และจะมีการส่งค่ากลับไปที่ฟังก์ชันเป็น FALSE

ค่า parameter ตัวที่ 3 เป็นค่าข้อมูลที่ต้องการจะส่งไปให้กับ kernel-mode driver เพื่อติดต่อหรือส่งไปเพื่อสั่งงานอุปกรณ์ฮาร์ดแวร์ โดยข้อมูลที่ส่งไปจะเป็นการส่งค่าพื้นที่ในหน่วยความจำที่เก็บค่าข้อมูลนั้นไว้

ค่า parameter ตัวที่ 4 เป็นค่าขนาดของข้อมูลที่ต้องการส่งไปให้ kernel-mode driver โดยส่วนมากค่านี้จะเป็นค่าตัวเลข

ค่า parameter ตัวที่ 5 เป็นค่าของพื้นที่ในหน่วยความจำที่ทำการส่งไปเพื่อใช้ในการรับค่าของข้อมูลที่จะส่งกลับมาจาก kernel-mode driver หรือเป็นค่าที่ส่งกลับมาจากอุปกรณ์ฮาร์ดแวร์ที่ส่งค่ากลับมาให้กับ user-mode driver โดยส่งผ่าน kernel-mode driver

ค่า parameter ตัวที่ 6 เป็นค่าขนาดของข้อมูลที่ได้รับกลับมาจาก kernel-mode driver โดยส่วนมากค่านี้จะเป็นค่าตัวเลข

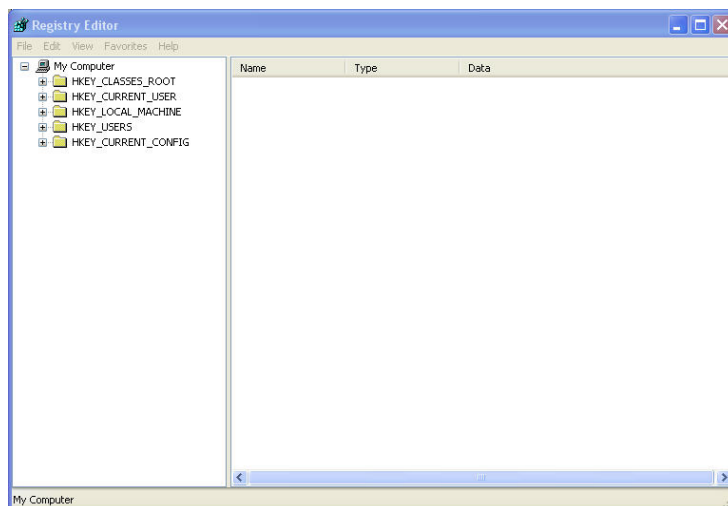
2.2.11.2 ฟังก์ชัน CreateFile()

เป็นการเรียกใช้ฟังก์ชันมาตรฐาน ของ Microsoft Win32 ซึ่งเป็นฟังก์ชันที่ทำงานบน user-mode driver และการทำงานจะส่ง IRP_MJ_CREATE ไปให้กับ kernel-mode driver เพื่อทำการสร้างไฟล์ Handle ของอุปกรณ์ฮาร์ดแวร์ เพื่อไว้สำหรับให้ user-mode driver ใช้ในการอ้างอิงถึงอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อด้วย รูปแบบการใช้งานมีดังนี้

```
Handle = CreateFile("\\\\.\\IOCTL", GENERIC_READ | GENERIC_WRITE,
    0, NULL, OPEN_EXISTING, flags, NULL);
if (Handle == INVALID_HANDLE_VALUE)
    <error>
...
CloseHandle(Handle);
```

ค่าที่ได้รับกลับมาจากฟังก์ชันนี้คือค่า Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อด้วย ถ้าหากสามารถทำการติดต่อกับอุปกรณ์ฮาร์ดแวร์ได้ก็จะได้รับค่า Handle ของอุปกรณ์นั้นกลับมา แต่ถ้าหากไม่สามารถติดต่อกับอุปกรณ์ฮาร์ดแวร์ได้ค่าที่จะได้รับกลับมาก็คือค่า INVALID_HANDLE_VALUE และค่าที่ทำการส่งไปมีทั้งหมด 7 ตัว แต่ค่าหลักๆ ที่จำเป็นต้องรู้คือ

ค่า parameter ตัวที่ 1 คือชื่อของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อด้วย ในที่นี้คือ IOCTL และพาธของการ Create File คือ [\\.](#) จะเป็นรูทของระบบเช่น C:\windows\system32 ระบบปฏิบัติการจะทำการแปลงเป็น \device\ HarddiskVolume1\Partition0 แต่เราไม่สามารถที่รู้ได้ว่าผู้ใช้ทั่วไปลงระบบปฏิบัติไว้ที่ใด จึงค่าเป็นรูทของระบบซึ่งจะมี Symboliclink ที่จะทำการเชื่อมโยงไปยัง Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อด้วยได้ หากเราต้องการตรวจสอบว่า Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อด้วยของเราได้ใส่เข้าไปในระบบหรือยัง ก็สามารถทำการตรวจสอบได้โดยการเข้าไปดูที่ Registry ของระบบปฏิบัติการ Windows เข้าไปที่ Run... พิมพ์คำว่า regedit กด OK จากนั้นเราก็จะเข้าสู่หน้าต่าง Registry Editor ของระบบปฏิบัติการ Windows ดังรูปที่ 2.11

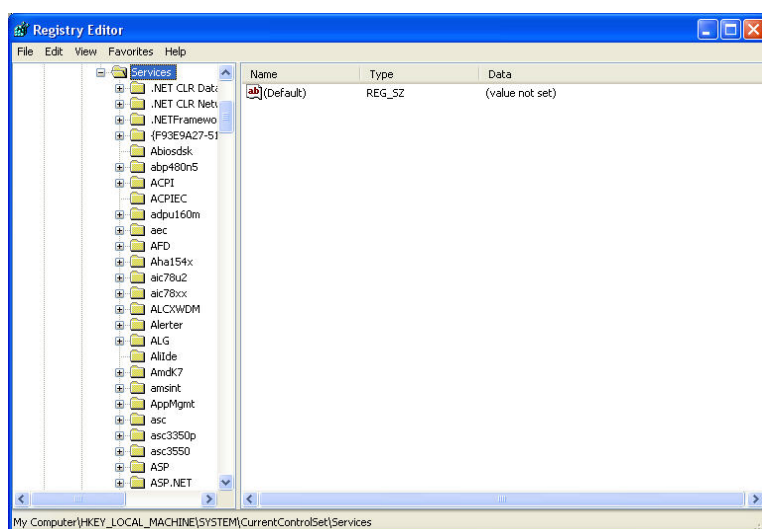


รูปที่ 2.11 หน้าต่าง Registry Editor

หากเราต้องการตรวจสอบว่า Device name ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดตั้งของเราได้ใส่เข้าไปในระบบหรือยัง ให้เข้าไปดูที่พาธ

RegistryPath = HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services

ดังรูปที่ 2.12 จะแสดง Device name ของอุปกรณ์ฮาร์ดแวร์ ที่มีอยู่ภายในเครื่อง ที่พาธ HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services ซึ่งจะมี Device name ของอุปกรณ์ฮาร์ดแวร์หลายชนิด ที่ได้ทำการติดตั้ง Device Driver ลงไปในเครื่องระบบปฏิบัติการ Windows



รูปที่ 2.12 พาธ HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services

ค่า parameter ตัวที่ 2 เป็นการบอกกับระบบปฏิบัติการ windows ว่าต้องเปิดไฟล์ Handle นี้เพื่อที่จะอ่านข้อมูลจากอุปกรณ์ฮาร์ดแวร์หรือเพื่อที่จะเขียนข้อมูลยังอุปกรณ์ฮาร์ดแวร์ ในที่นี้ใช้เปิดเพื่อทั้งอ่านและเขียน จึงใช้ operator OR เข้ามาช่วย

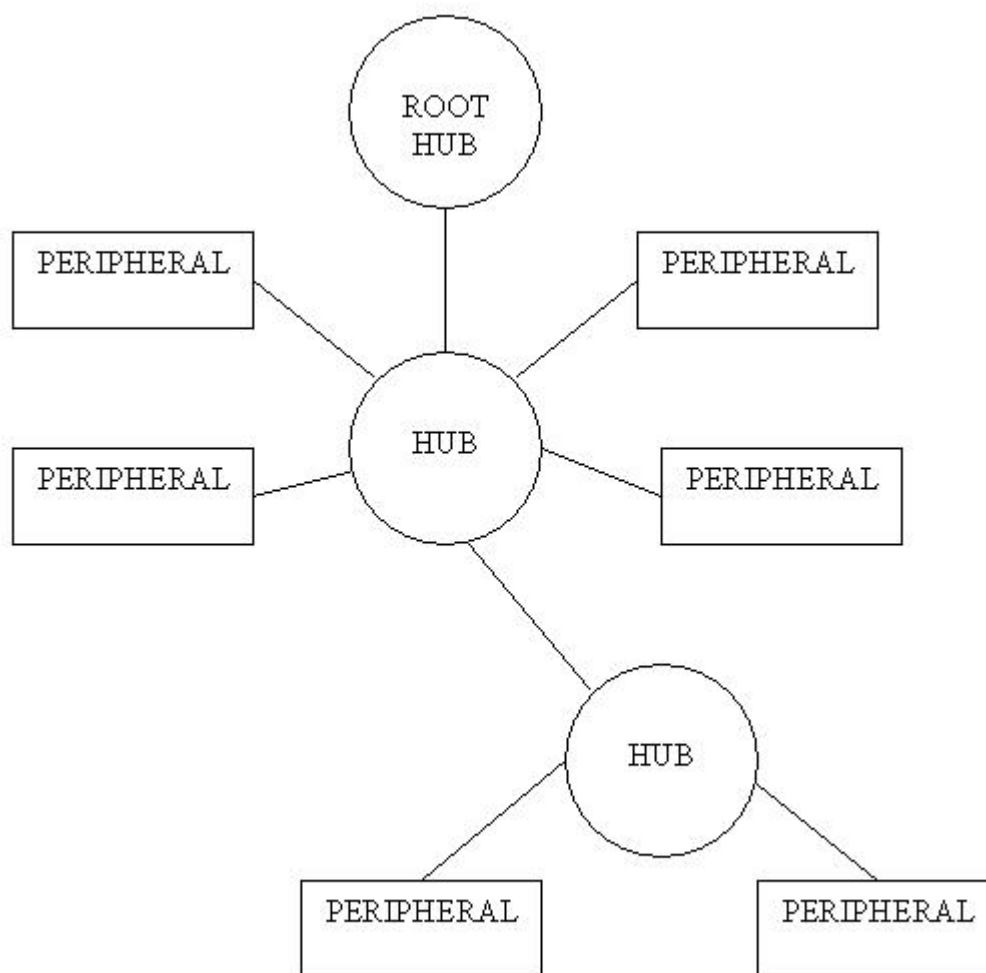
ส่วนถัดมาจากการ CreateFile() คือการตรวจสอบว่าสามารถสร้างไฟล์ Handle ของอุปกรณ์ฮาร์ดแวร์ที่ต้องการจะติดต่อได้หรือไม่ ถ้าหากสามารถทำการติดต่อกับอุปกรณ์ฮาร์ดแวร์ได้ก็จะได้รับค่า Handle ของอุปกรณ์ฮาร์ดแวร์นั้นกลับมา แต่ถ้าหากไม่สามารถติดต่อกับอุปกรณ์ฮาร์ดแวร์ได้ค่าที่จะได้รับกลับมาก็คือค่า INVALID_HANDLE_VALUE เราสามารถใช้ if ตรวจสอบเช็คค่าได้

เมื่อทำการ CreateFile() ได้สำเร็จ ก็ทำการรับ – ส่งข้อมูลกับอุปกรณ์ฮาร์ดแวร์ตามปกติ โดยใช้ไฟล์ Handle ที่ได้จากการ CreateFile() ในการอ้างอิงกับอุปกรณ์ฮาร์ดแวร์ที่จะติดต่อด้วย จากนั้นเมื่อการทำงานเสร็จสิ้นก็จะใช้ฟังก์ชัน CloseHandle() ในการคืนค่าไฟล์ Handle ให้กับระบบโดยค่า parameter ที่ส่งไปก็คือค่าไฟล์ Handle ที่ได้สร้างขึ้นมาจากฟังก์ชัน CreateFile ในตอนแรก

2.3.ทฤษฎีเบื้องต้นส่วนของ USB

2.3.1 Bus topology

Topologyบนbusจากรูปจะเห็นได้ว่าที่จุดกลางของแต่ละstarจะเป็นhub ซึ่งแต่ละจุดบน star เป็น device ที่ต่ออยู่กับhub ซึ่งจำนวนของpoint บนแต่ละ star เปลี่ยนแปลงได้ โดยเราสามารถเพิ่มจำนวนport ได้ ซึ่งทุกdevices บน bus จะต้องแบ่งกันใช้ data path ที่จะใช้เพราะมีอยู่เส้นเดียวเท่านั้น



รูปที่ 2.13 USB ใช้ Topology แบบ Star มีHub เป็นศูนย์กลาง

2.3.2.Host Duties

2.3.2.1.Detect Device

เมื่อเราเปิดเครื่อง hub จะบอกให้ host ทราบว่ามี Device ติดต่ออยู่ซึ่งในprocess นี้เรียกว่าการenumurate โดยที่ host จะ assign address และ request ข้อมูลจากแต่ละ device

2.3.2.2. Manage data flow

Host จะเป็นตัวจัดการflowของdata เมื่ออุปกรณ์หลายตัวต้องการที่จะ transferข้อมูลในเวลาเดียวกัน host controllerจะจัดการ โดยแบ่ง data path ละ 1 millisecond ให้แต่ละport ของแต่ละ frame

2.3.2.3. Error Checking

Host มีหน้าที่ในการcheck error มันเพิ่ม error checking bit ลงไปใน data ที่ส่งไป เมื่อ device driver ได้รับdata มันจะสามารถประมวลผลและเปรียบเทียบ ถ้าผลลัพธ์ที่ได้mathกัน device จะไม่ ack ข้อมูลที่รับเข้ามาและ host จะรู้ว่าควรทำการส่งซ้ำอีกครั้ง

2.3.3. Peripheral Duties

ในหลากหลายวิธี peripheral หรือ device มีหน้าที่เป็นเหมือนกระจกที่สะท้อน host เมื่อ host นั้น initial การติดต่อกับ peripheral นั้นต้อง respond กับ device ไม่สามารถติดต่อกับ usb ด้วยตัวมันเอง มันต้องรอและ respond เมื่อ host ติดต่อมา ซึ่ง USB Controller ในdevice จะเป็นตัวจัดการของตัวมันเองโดยอัตโนมัติ

2.3.3.1. Detect Communication

แต่ละ device จะคอยจับตาดู device address ในแต่ละการติดต่อบนbus ถ้าaddressไม่math กับ address ที่เก็บไว้ มันจะไม่สนใจ แต่ถ้า address นั้น mathกัน deviceจะเก็บdataลงในbuffer และ generate interrupt สัญญาณไปบอกว่า data ส่งมาถึงแล้ว

2.3.3.2. Respond

เมื่อเปิดเครื่องหรือเมื่อdevice attaches กับ power system device ต้อง respond ต่อ request ที่ host สร้างใน enumerate process ซึ่ง USB ทุกตัวต้อง respond

2.3.4. Device Endpoints

การส่งข้อมูลทั้งหมดจะเดินทางไปหรือมาจาก device endpoint ซึ่งendpoint เป็น bufferซึ่งเก็บข้อมูล โดยทั่วไปมันจะเป็นblockของข้อมูล หรือ register ที่อยู่ใน chip controller ซึ่งข้อมูลที่ถูเก็บไว้ที่ endpoint อาจเป็นข้อมูลที่รับมา หรือ ข้อมูลที่รอส่ง โดยที่ host มีbufferสำหรับรับข้อมูลและข้อมูลที่พร้อมจะส่ง แต่hostไม่มีendpoint แทนที่มันจะทำงานเป็นจุดเริ่มสำหรับการติดต่อกับdevice endpoint

รายละเอียดที่กำหนดdevice endpoint ซึ่ง addressของportจะเป็นaddressที่ไม่ซ้ำกับใครเอาไว้ติดต่อกันระหว่าง host กับ device โดยที่addressของแต่ละ endpointประกอบด้วย endpoint number และ direction

โดยnumberอาจอยู่ในระหว่าง 00-0Fh. ส่วนdirection เป็นภาพที่เรามองจากhost ซึ่งจะมี IN และ OUT โดยที่ IN จะเป็นทิศทางที่เข้าสู่ host และ OUTจะเป็นทิศทางที่ออกจาก host เสมอ ที่endpoint จะถูกกำหนดโดยใช้ control transfer ในการส่งข้อมูลทั้ง INและOUT ซึ่งต้องแบ่งendpoint number

ทุกdevice ต้องมี Endpoint 0 ไว้กำหนดendpoint ซึ่งการส่งข้อมูลแบบอื่นจะเป็นแบบทิศทางเดียวเท่านั้น endpointหนึ่งสามารถเป็นได้ทั้ง INและOUT ตัวอย่างเช่น endpoint 1 สามารถเป็น IN endpoint address พร้อมกับเป็นOUT endpoint address

2.3.5.Pipes

ก่อนจะสามารถเกิดการส่งข้อมูลได้นั้น hostและ deviceต้องสร้างpipe ขึ้นมาเสียก่อน pipe ไม่ได้เป็น physical object แต่มันเป็นการรวมกันระหว่าง device endpointและ host controller software

Host จะสร้างpipe หลังจากทีระบบถูกบูทขึ้นมา หรือ device มีการทำ attachment ถ้าdeviceถูกนำเอาออกไปจากบัส hostจะนำเอาpipe ที่ไม่จำเป็นออกตามไปด้วย หรือ host อาจจะไปร้องขอให้สร้าง pipe ตัวใหม่ขึ้นมา หรือเอาออกในเวลาต่อมาก็ได้

รายละเอียดของข้อมูลที่ได้รับที่host จะประกอบด้วยdescriptor สำหรับแต่ละendpoint ซึ่งแต่ละ endpoint descriptor จะเป็น blockของข้อมูลซึ่งบอกhostว่า อะไรที่มันจำเป็นต้องรู้เกี่ยวกับendpointในการติดต่อกับมัน ซึ่งจะประกอบไปด้วย endpoint address ,ชนิดของการส่งข้อมูล และขนาดของpacket สูงสุด

ในบางกรณี hostจะสร้างpipeเดียวเท่านั้นหลังจากที่แน่ใจว่าbusมีbandwidthพอที่จะทำการส่งข้อมูลภายในเวลาที่requestมา กรณีนี้จะทำเมื่อเราใช้ isochronous transfer

ในบางกรณีhostจะตรวจสอบ bandwidthดูก่อนสร้าง pipe ถ้าbandwidthพอกับที่ต้องการ hostจะรับrequestและทำการส่งข้อมูลแต่ถ้า bandwidthไม่พอ hostจะปฏิเสธrequestที่จะสร้างpipe และ software ต้องrequestเข้ามาใหม่อีกครั้งหนึ่ง หรือไม่ก็จะรอจนกว่าbandwidthจะพอกับที่ต้องการ

2.3.6.ชนิดของการส่งข้อมูล

USBนั้นถูกออกแบบให้มีhandleอยู่หลายชนิดในการจัดการกับ peripherals ที่มีความต้องการสำหรับ transfer rate , response time และ error correcting ซึ่งทั้ง4 ชนิดของการส่งข้อมูลแต่ละความจำเป็นที่แตกต่างกันและอุปกรณ์ที่รองรับกับชนิดของการส่งข้อมูลซึ่งต้องเหมาะสมกับวัตถุประสงค์ ซึ่งสามารถแบ่งออกเป็น4 ชนิดดังนี้

2.3.6.1.Control transfers

Control transfers เป็นชนิดเดียวเท่านั้นที่มีfunctionในการกำหนดรายละเอียดของUSB การส่งข้อมูลแบบนี้จะทำให้host สามารถอ่านและเลือกรูปแบบและsetอย่างอื่นได้ในการenumerateซึ่ง deviceทุกตัวต้องรองรับcontrol transfer

2.3.6.2.Bulk transfers

Bulk transfers เป็นวิธีที่ใช้ในสถานการณ์ที่ไม่เกิดการคับคั่งของเส้นทาง อย่างเช่นการส่งไฟล์ไปที่เครื่องพิมพ์ หรือการรับข้อมูลจากเครื่องสแกนเนอร์ ในกรณีนี้เป็นวิธีที่มีการส่งข้อมูลได้รวดเร็ว แต่ข้อมูลสามารถถูกพักไว้ก่อนได้ถ้าจำเป็น

2.3.6.3.Interrupt transfers

Interrupt transfers ถูกนำไปใช้สำหรับ device ที่ต้องรับ attention อย่างรวดเร็วจาก host หรือ device ซึ่ง Interrupt transfers เป็นวิธีเดียวเท่านั้นที่สามารถใช้ใน low-speed device อย่างเช่น keyboard หรือ mouse ที่สามารถใช้ Interrupt transfers ในการส่ง keypress หรือ ตำแหน่งของ mouse ที่เคลื่อนที่ไป

2.3.6.4.Isochronous transfers

Isochronous transfers ถูกนำไปใช้สำหรับ device ที่ต้องส่งข้อมูลที่อัตราคงที่ อย่างเช่น การส่งไฟล์เสียงเพื่อใช้การทำแบบ real time ซึ่งต้องมีอัตราคงที่

2.3.7.รูปแบบการส่งข้อมูลที่ใช้ในโปรแกรม

Bulk Transfers

Bulk Transfers ประกอบด้วยมากกว่าหนึ่ง in หรือ out transaction ซึ่ง Bulk Transfers เป็นแบบ one – way ซึ่ง action ใน transactions ถ้าเป็น IN ก็ต้องเป็น IN ทั้งหมด หรือถ้าเป็น OUT ก็ต้องเป็น OUT ทั้งหมด

Data Size

Bulk Transfers สามารถมี Maximum Packet ได้ 8,16,32 ถึง 64 bytes ในระหว่างการ enumerate นั้น host จะดูว่ามี maximum packet เท่าไรจาก device ซึ่งข้อมูลนั้นอาจจะมีขนาดเล็กกว่าหรือเท่ากันหรืออาจจะมีขนาดใหญ่กว่า maximum ถ้ากลุ่มข้อมูลมีขนาดมากกว่า host controller จะแบ่งไปเป็น multiple transaction และ low overhead

2.3.8. Stream และ Message Pipes

ในการแบ่ง pipe ออกเป็นชนิดของการส่งข้อมูล USB ได้กำหนด pipe เป็น stream หรือ message type ซึ่ง control transfer เป็นวิธีเดียวที่ใช้ message pipe ทั้งสองด้าน ส่วนวิธีอื่นจะเป็น stream pipe ทั้งหมด

ใน message pipe แต่ละครั้งจะเริ่มที่ Setup transaction ที่บรรจุ request มา เมื่อส่งเสร็จแล้ว host และ device อาจแลกเปลี่ยนข้อมูลและข้อมูลสถานะกัน ส่วนใน stream pipe นั้น

ข้อมูลจะไม่มีรูปแบบที่กำหนดตายตัว ซึ่ง device จะรับข้อมูลอะไรก็ตามที่ส่งมา แล้ว device firmware หรือ host จะเป็นคนประมวลผลข้อมูลนั่นเอง

2.3.9. Enumeration

ก่อนที่ applications จะสามารถติดต่อกับ device ได้ นั้น host จำเป็นต้องเรียนรู้เกี่ยวกับชนิดของการส่งข้อมูล และ endpoint ที่ device นั้นรองรับอยู่ ดังนั้น host จะต้องกำหนด address ให้กับ device แล้ว host สามารถแลกเปลี่ยนข้อมูลกันได้ เรียกว่า enumeration

ระบบการทำงานของ การ Enumeration อย่างแรกก็คือ หน้าที่ของ hub ที่จะเป็นคนตรวจหาว่ามี device ที่เข้ามาในระบบ หรือถูกนำออกไป ซึ่งในแต่ละ hub มี IN pipe ไว้สำหรับรายงานเหตุการณ์ไปที่ host เมื่อระบบถูกบูทขึ้นมา host จะสำรวจ root hub ถ้ามี device ใดติดต่อเข้ามาใหม่ และจะทำอย่างนี้ต่อไปเป็นระยะๆ เพื่อจะดูว่ามี device ตัวไหนที่ถูก attachment หรือ remove ออกไป

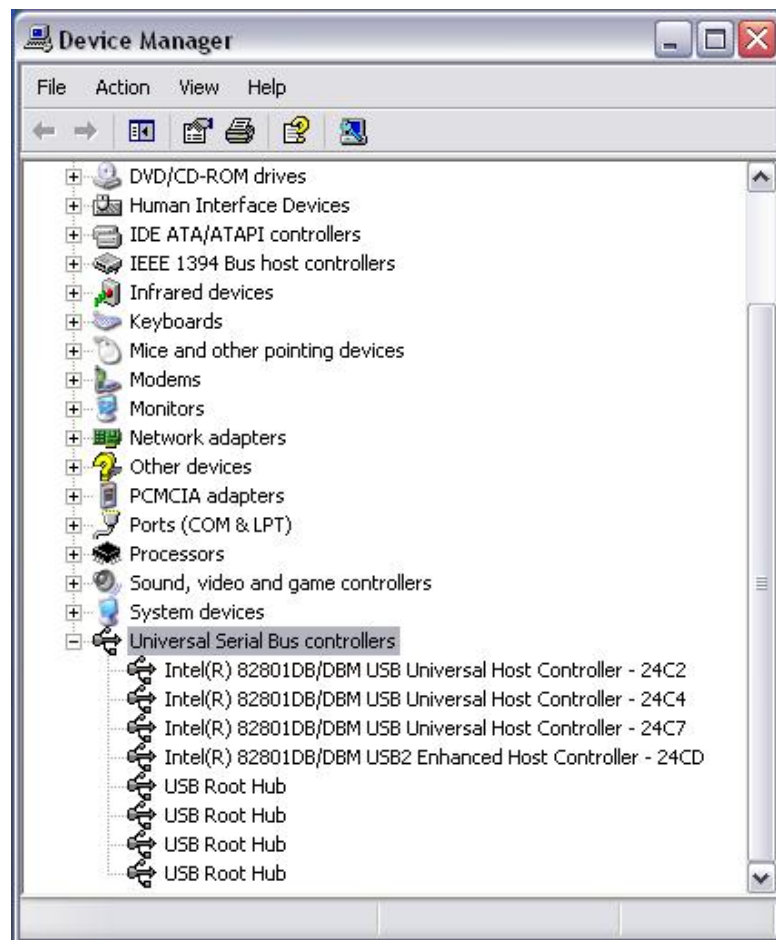
ในการเรียนรู้ device ตัวใหม่ host จะส่ง request ไปที่ device hub ทำให้ hub นั้นสร้างเส้นทางการติดต่อระหว่าง host และ device ซึ่ง host จะพยายาม ระบุ device โดยการที่จะ enumerate นั้นจะต้องกำหนดค่าเริ่มต้นแลกเปลี่ยนข้อมูลทำให้ host ได้ device driver ที่ติดต่อกับ device ได้ โดยการทำงานของมันนั้นจะประกอบไปด้วยการกำหนด address ของ device อ่านข้อมูลจาก device กำหนดค่าและโหลด device driver ขึ้นมาและเลือกข้อกำหนดต่างที่ระบุไว้ ดังนั้น device ที่ถูกกำหนดและพร้อมที่จะ transfer

ข้อมูลโดยใช้ endpoint ใดๆในที่เรากำหนด

Host จะ enumerate โดยส่ง control transfer ที่ใช้ endpoint 0 ซึ่งทุก device จะต้องรองรับ control transfer นี้ เมื่อ enumerate เสร็จแล้ว device นั้นต้องตอบแต่ละ request โดย return กลับไปหาและทำงานตาม request นั้น

จากมุมมองของผู้ใช้นั้น ไม่ควรจะให้เห็นเกี่ยวกับการ enumerate และควรทำงานอย่างอัตโนมัติ สำหรับในบางกรณีที่ windows บอกว่า ตรวจพบ device ตัวใหม่ และในบางครั้งถ้าเป็นการใช้ครั้งแรก ผู้ใช้ควรจะเตรียมแผ่นซึ่งบรรจุไฟล์ INF และ device driver ไว้ด้วย

เมื่อทำการ enumeration เสร็จสมบูรณ์แล้ว windows จะเพิ่ม device ตัวใหม่ไปไว้ใน Device Manager ที่แสดงอยู่ใน Control Panel ดังรูปที่ 2.14



รูปที่ 2.14 Device Manager ใน Windows แสดง device ที่ตรวจพบ

ขั้นตอนการenumeration

1. ให้ผู้ใช้ทำการเสียบอุปกรณ์เข้าที่พอร์ต USB
2. hub นั้นจะทำการตรวจหา device
3. host เรียนรู้ device ที่เข้ามาใหม่
4. hub จะ reset ตัว device
5. hub สร้างเส้นสัญญาณระหว่าง device และ bus
6. hub ตรวจหาความเร็วของ device
7. host ส่ง request ไปหา maximum packet size ของ pipe
8. host ทำการกำหนดค่า address
9. host เรียนรู้เกี่ยวกับความสามารถของ device
10. host จะ assigns และ loads ตัว device driver
11. device driver ทำการเลือก configuration

2.4 การ Install Device Driver

ในการ Install Device Driver จะใช้ไฟล์ที่มีนามสกุล .inf เราเรียกไฟล์ชนิดนี้ว่าไฟล์ INF เราจะใช้ไฟล์ชนิดนี้ในการ install device driver ซึ่งรายละเอียดภายในไฟล์นี้จะมีการระบุข้อมูลต่างๆของอุปกรณ์ฮาร์ดแวร์ และมีรหัสของอุปกรณ์ด้วย รายละเอียดมีดังนี้

```
[Version]

..

Class=USB

..

[DestinationDirs]

DefaultDestDir=10,System32\Drivers


[SourceDisksFiles]

test.sys=1


[SourceDisksNames]

1=%INSTDISK%,,,,objchk\i386


[DeviceList]

%DESCRIPTION%=DriverInstall,

USB\VID_????&PID_????&REV_????[DriverInstall.NT]

CopyFiles=DriverCopyFiles


[DriverCopyFiles]

test.sys,,,

test.jnk,,,


[Strings]

INSTDISK="Project Disc"

DESCRIPTION="Test Project"
```

จากตัวอย่างไฟล์ INF ข้างบนมีการกำหนด class จะเป็นการแบ่งประเภทของอุปกรณ์ฮาร์ดแวร์ ซึ่งมีอยู่หลายประเภทดังแสดงในตาราง

ตารางที่ 2.1 Device classes ที่ใช้ภายในไฟล์ INF

ชื่อ Class	คำอธิบาย
1394	IEEE 1394 host bus controllers (but not peripherals)
Battery	Battery devices
CDROM	อุปกรณ์จำพวก CD ROM
DiskDrive	อุปกรณ์ฮาร์ดดิสก์
Display	การ์ดแสดงผล
FDC	Floppy disk controllers
FloppyDisk	อุปกรณ์ฟลอปปีดิสก์
HDC	Hard disk controllers
HIDClass	Human input devices
Image	Still-image capture devices, including cameras and scanners
Infrared	NDIS miniport drivers for Serial-IR and Fast-IR ports
Keyboard	อุปกรณ์คีย์บอร์ด
MediumChanger	SCSI media changer devices
Media	Multimedia devices, including audio, DVD, joysticks, and full-motion video capture devices
Modem	โมเด็ม
Monitor	มอนิเตอร์
Mouse	อุปกรณ์เมาส์
MTD	Memory technology driver for memory devices
Multifunction	อุปกรณ์หลายฟังก์ชันในเครื่องเดียว
MultiportSerial	Intelligent multiport serial cards
Net	การ์ดแลน
NetClient	Network file system and print providers (client side)
NetService	Server-side support for network file systems
NetTrans	Network protocol drivers

ตารางที่ 2.1 Device classes ที่ใช้ภายในไฟล์ INF (ต่อ)

ชื่อ Class	คำอธิบาย
PCMCIA	PCMCIA and CardBus host controllers (but not peripherals)
Ports	อุปกรณ์ พอร์ตขนานและพอร์ตอนุกรม
Printer	ปริ้นเตอร์
SCSIAdapter	SCSI and RAID controllers, host bus adapter miniports, and disk array controllers
SmartCardReader	Smart card readers
System	System devices
TapeDrive	เทปไดรฟ์
USB	USB host controllers and hubs (but not peripherals)
Volume	Logical storage volume drivers

ถัดจากการกำหนด Class ที่อยู่ในส่วนของ [Version] ก็จะเป็นการกำหนดค่าในส่วนของ [DestinationDirs] เป็นการกำหนดว่าจะให้ทำการ install device driver แล้วนำไฟล์ .sys ที่เป็นไฟล์ driver ไปเก็บไว้ที่พาทไหน ในที่นี้ให้นำไปเก็บไว้ที่ พาท C:\WINDOWS\system32\drivers กรณีที่มีการลงระบบปฏิบัติการ windows ไว้ที่ ไดรฟ์ C แต่เราไม่สามารถรู้ได้ว่าผู้ใช้ได้ลงระบบปฏิบัติการ windows ไว้ที่ พาทไหน เราจึงใช้ตัวแปรระบบ 10 ซึ่งจะเป็นการอ้างอิงถึงไดรฟ์ที่ผู้ใช้ลงระบบปฏิบัติการ windows ไว้แล้วไปที่ไดเรกทอรี WINDOWS

จากนั้นจะเป็นส่วนของ [SourceDisksFiles] เป็นส่วนที่ระบุว่า ไฟล์ไดรฟ์เวอร์ชื่ออะไร ในส่วนของ [SourceDisksNames] เป็นการระบุว่าไฟล์ไดรฟ์เวอร์เก็บไว้ในไดเรกทอรีอะไร ในดิสก์ชื่อว่าอะไร ค่า %INSTDISK% เป็นตัวแปรที่บอกถึงชื่อดิสก์ เนื่องจากการ install device driver จะทำโดยผ่านทางดิสก์ที่มาจากผู้ผลิตอุปกรณ์ฮาร์ดแวร์ และค่า %INSTDISK% จะมีการกำหนดในอยู่ในส่วนท้ายของไฟล์ INF จะอยู่ในส่วนของ String ส่วนของ [DeviceList] เป็นการบอกถึงรายละเอียดของอุปกรณ์ฮาร์ดแวร์ ซึ่งจะมีการกำหนดรหัสของอุปกรณ์ฮาร์ดแวร์ที่มาจากบริษัทผู้ผลิตด้วย ซึ่งจะอธิบายอยู่ในส่วนของ Device Identifiers ส่วน [DriverCopyFiles] จะเป็นการกำหนดว่าให้ทำการคัดลอกไฟล์ใดบ้างไปไว้ที่พาท C:\WINDOWS\system32\drivers กรณีที่มีการลงระบบปฏิบัติการ windows ไว้ที่ ไดรฟ์ C ส่วน [Strings] เป็นการกำหนดค่าตัวแปร string ที่ใช้ภายในไฟล์ INF นี้

2.4.1 Device Identifiers

ในอุปกรณ์ฮาร์ดแวร์ทุกชนิดที่มีการทำงานเป็นแบบ Plug and Play คือ อุปกรณ์ที่นำมาเสียบต่อเข้ากับเครื่องคอมพิวเตอร์แล้วก็ติดตั้ง device driver ก็สามารถใช้ได้เลย อุปกรณ์ฮาร์ดแวร์ที่มีการทำงานเป็นแบบ Plug and Play นี้จะมีรหัสของอุปกรณ์ซึ่งจะมีค่าไม่ซ้ำกันโดยการกำหนดค่ารหัสของอุปกรณ์นี้ในแต่ละชนิดของอุปกรณ์ก็จะมีการกำหนดค่าที่ไม่เหมือนกัน โดยจะมีการกำหนดค่าและแบ่งชนิดดังนี้

2.4.1.1 อุปกรณ์ PCI

มีรูปแบบการกำหนดดังนี้

```
PCI\VEN_vvvv&DEV_dddd&SUBSYS_ssssssss&REV_rr
```

vvvv คือ รหัสผู้ผลิต

dddd คือ รหัสอุปกรณ์

ssssssss คือ รหัสย่อยของอุปกรณ์ ส่วนมากจะเป็น 0

rr คือ รหัสการแก้ไขปรับปรุง

2.4.1.2 อุปกรณ์ PCMCIA

มีรูปแบบการกำหนดดังนี้

```
PCMCIA\Manufacturer-Product-Crc
```

Manufacture คือ ชื่อผู้ผลิต

Product คือ ชื่ออุปกรณ์

Crc คือ เลขฐาน 16 จำนวน 4 บิต เป็นค่า CRC checksum ของอุปกรณ์

2.4.1.3 อุปกรณ์ SCSI

มีรูปแบบการกำหนดดังนี้

```
SCSI\ttttvvvvvvvvpppppppppppppppprrrr
```

tttt คือ โค้ดชนิดของอุปกรณ์

vvvvvvvv คือ ชื่อผู้ผลิต โดยแสดงเป็นตัวอักษร 8 ตัว

pppppppppppppppppppppp คือ ชื่อรหัสของอุปกรณ์ โดยแสดงเป็นตัวอักษร 16 ตัว

rrrr คือ รหัสการแก้ไขปรับปรุง

2.4.1.4 อุปกรณ์ IDE

มีรูปแบบการกำหนดดังนี้

```
IDE\tttvpvprrrrrrrrr
IDE\vpvprrrrrrrrr
IDE\tttvpvp
vpvprrrrrrrrr
gggg
```

tttt คือ โค้ดชนิดของอุปกรณ์

vpvp คือ ชื่อผู้ผลิตแล้วตามด้วยชื่อของอุปกรณ์ โดยค้นด้วยเครื่องหมาย underscore (_)

rrrrrrr คือ รหัสการแก้ไขปรับปรุง โดยแสดงเป็นตัวอักษร 8 ตัว

gggg คือ โค้ด generic ที่แสดงชนิดของอุปกรณ์

2.4.1.5 อุปกรณ์ USB

มีรูปแบบการกำหนดดังนี้

```
USB\VID_vvvv&PID_dddd&REV_rrrr
```

vvvv คือ เลขฐาน 16 จำนวน 4 บิต เป็นรหัสผู้ผลิต

dddd คือ เลขฐาน 16 จำนวน 4 บิต เป็นรหัสของอุปกรณ์

rrrr คือ รหัสการแก้ไขปรับปรุง